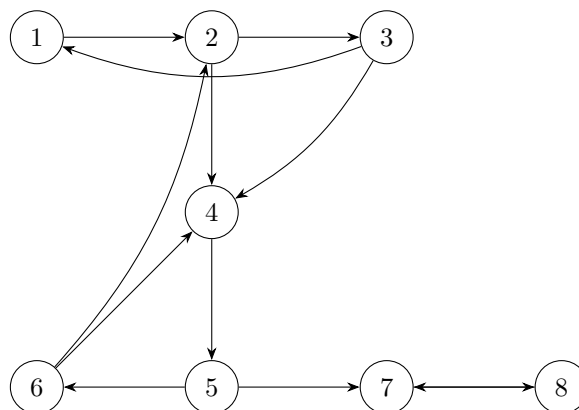

Handing in and marking

For this assignment, you need to submit your solutions to the pencil-and-paper exercises on Crowdmark and your solutions to the programming question on MarkUs. Your pencil-and-paper solutions will be marked with respect to correctness, clarity, brevity, and readability. Your code will be marked with respect to correctness, efficiency, program design and coding style, clarity, and readability. This exercise counts for 10% of the course grade.

Question 1. Strongly Connected Components (SCC) [10 MARKS]

1. Consider the directed graph G below (vertices $1 \dots 8$).



Run the algorithm discussed in class on G . Show:

- First DFS finishing times (give the order).
- The graph G^T .
- The second DFS forest on G^T in the first-pass finishing-time order.
- The final SCCs.

2. A partition of a set S is a collection of subsets $S_1, S_2, \dots, S_n$ such that

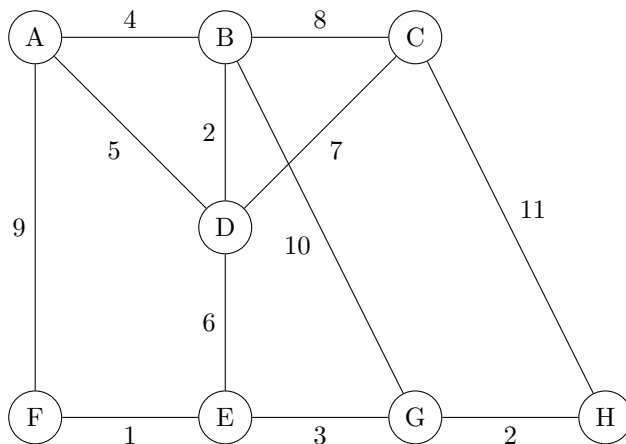
$$S_1 \cup S_2 \cup \dots \cup S_n = S, \quad \text{and} \quad S_i \cap S_j = \emptyset \text{ for all } i \neq j.$$

Prove that the SCCs of a directed graph form a partition of the vertex set.

3. Let G be a directed graph. We define the *condensation graph* G_{SCC} of G by contracting each SCC of G into a single node, keeping edges between components if any edge exists in G between their vertices. Prove that G_{SCC} is a Directed Acyclic Graph (DAG).
4. Suppose you add a single directed edge to G . Give tight conditions under which the number of SCCs:
- (a) decreases,
 - (b) stays the same, or
 - (c) can never increase.

Question 2. Minimum Spanning Tree (MST) [10 MARKS]

Consider the undirected, connected weighted graph below (vertices $A \dots H$). Edge weights are indicated on the edges.



1. Run *Kruskal's algorithm*. List edges in non-decreasing weight order, mark which are chosen/skipped, and show the clusters after each chosen edge. Give the final MST and its total weight.
2. Run *Prim's algorithm* starting from vertex A . Show the priority queue at each step. Provide the same final MST and total weight.

Question 3. Minimum Spanning Tree - 2 [10 MARKS]

Let $G = (V, E)$ be an undirected, connected graph whose weight function is $w : E \rightarrow \mathbb{R}$, and suppose that $|E| \geq |V|$ and all edge weights are distinct. We define a second-best minimum spanning tree as follows. Let $\mathcal{T}$ be the set of all spanning trees of G , and let T' be a minimum spanning tree of G . Then a *second-best minimum spanning tree* is a spanning tree T such that

$$w(T) = \min_{T'' \in \mathcal{T} \setminus \{T'\}} \{w(T'')\}.$$

1. Show that the minimum spanning tree is unique, but that the second-best minimum spanning tree need not be unique.
2. Let T be the minimum spanning tree of G . Prove that G contains edges $(u, v) \in T$ and $(x, y) \notin T$ such that

$$T - \{(u, v)\} \cup \{(x, y)\}$$

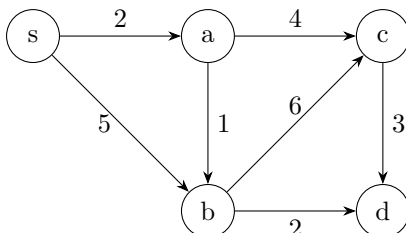
is a second-best minimum spanning tree of G .

3. Let T be a spanning tree of G , and for any two vertices $u, v \in V$, let $\max[u, v]$ denote the edge of maximum weight on the unique simple path between u and v in T . Describe an $O(V^2)$ -time algorithm that, given T , computes $\max[u, v]$ for all $u, v \in V$.
4. Give an efficient algorithm to compute the second-best minimum spanning tree of G .

Question 4. Shortest Path Algorithm [10 MARKS]

Let $G = (V, E)$ be a directed graph with non-negative edge weights $w : E \rightarrow \mathbb{R}_{\geq 0}$, and let $s \in V$ be a specified source vertex. We denote by $\delta(s, v)$ the shortest-path distance from s to v .

1. Run Dijkstra's algorithm on the graph below starting from source s . Show the contents of the priority queue after each extraction (as done in class), and compute the final shortest-path distances $\delta(s, v)$ for all vertices.



2. Prove that when all edge weights are non-negative, every vertex is *finalized* (i.e., removed from the priority queue) exactly once during Dijkstra's algorithm.
3. Suppose some edge weight in the graph is negative. Give an explicit example showing that Dijkstra's algorithm may produce incorrect shortest-path distances. Your example should contain at most 4 vertices. What part of the proof (as discussed in class) fails when using negative weights?
4. We are given a directed graph $G = (V, E)$ on which each edge $(u, v) \in E$ has an associated value $r(u, v)$, which is a real number in the range $0 \leq r(u, v) \leq 1$ that represents the reliability of a communication channel from vertex u to vertex v . We interpret $r(u, v)$ as the probability that the channel from u to v will not fail, and we assume that these probabilities are independent. Give an efficient algorithm to find the most reliable path between two given vertices.

Question 5. Implementing Graph Algorithms [75 MARKS]

We provided you with the starter code for implementing an undirected graph and several algorithms for it. Your task is to implement the functions declared in `minheap.h`, `graph.h`, and `graph_algos.h` that are not already implemented in `minheap.c`, `graph.c`, and `graph_algos.c`. Note that `.c` files are the only files you will submit, so make sure your implementation works with the original provided `.h` files. Make sure to **carefully study the starter code** before you begin to add your own. The marking scheme for this question is as follows:

- Correctness:
 - MinHeap functions `extractMin`, `insert`, `decreasePriority`: 3 marks each
 - Graph functions required in `graph.h`: 6 marks
 - `Edge* getMSTprim(Graph* graph, int startVertex)`: 15 marks
 - `Edge* getDistanceTreeDijkstra(Graph* graph, int startVertex)`: 15 marks
 - `EdgeList** getShortestPaths(Edge* distTree, int numVertices, int startVertex)`: 15 marks
- Program design (modular implementation, self-explanatory code, clear logic, no repeated code, no unnecessary code): 10 marks
- Readability and coding style (good use of white space, good naming, etc.): 5 marks
 - You are encouraged (but not required) to use a linter to help check and/or auto-format your code.

The runtime complexity requirements for your functions are as follows ($n = |V|$ and $m = |E|$):

- `extractMin`, `insert`, `decreasePriority`: $\mathcal{O}(\log n)$ time.
- `getMSTprim`: $\mathcal{O}((n + m) \log n)$ time.
- `getDistanceTreeDijkstra`: $\mathcal{O}((n + m) \log n)$ time.